

Memory-Efficient Spiking Neural Network Training with Backpropagation-Free Learning Rules

Ertuğrul Keskin

Computer Engineering Department
Bogazici University
Istanbul, Türkiye
ertugrul.keskin1@std.bogazici.edu.tr

Arda Yurdakul

Computer Engineering Department
Bogazici University
Istanbul, Türkiye
yurdakul@bogazici.edu.tr

Abstract—Spiking neural networks (SNNs) are promising for energy-efficient learning on resource-constrained devices due to their sparse and event-driven computation. However, their mainstream training method, backpropagation through time (BPTT), requires storing temporal neuron states and intermediate activations across time steps, which leads to high training-memory consumption and limits on-device learning on memory-constrained systems. Reducing or completely removing backpropagation has become an important direction in the SNN literature over the last decade. This work proposes RATE-PEPITA, an SNN-compatible adaptation of Present the Error to Perturb the Input to Modulate Activity (PEPITA), which was originally designed as a backpropagation-free learning algorithm for ANNs. To further reduce memory requirements and boost accuracy, the temporal windowing concept in ANNs is adapted to SNNs. Moreover, a two-phase design space exploration method is proposed to find the optimal window size for the highest accuracy. The proposed methods are compared against BPTT and other backpropagation-free SNN learning rules. Using the analytical memory model proposed in this work, RATE-PEPITA is shown to reduce memory requirements by $10\times$ on average across diverse real-world datasets. It is also demonstrated that optimized temporal windowing boosts the accuracy of all examined methods, making some of them even outperform BPTT on some datasets.

Index Terms—spiking neural networks, memory-efficient learning, training memory analysis, temporal windowing, BPTT, E-PROP, Forward-Forward Algorithm, PEPITA

I. INTRODUCTION

Spiking neural networks (SNNs) process temporal data through sparse, event-driven spike communication inspired by biological neural systems [1]. Unlike conventional artificial neural networks (ANNs), SNNs transmit information through discrete spike events rather than dense continuous activations. This makes them attractive for efficient processing, particularly in resource-constrained settings [2].

Despite this promise, SNN training remains challenging. Backpropagation through time (BPTT) is commonly used by unfolding the network over time and propagating gradients through temporal dependencies [3]. However, BPTT requires neuron states and intermediate activations to be stored at every time step, causing training memory to scale with sequence length [4]. Recent work on resource-aware training [5] and strict-memory on-device learning [6] further shows that training algorithms must be designed around explicit memory constraints.

This makes the learning rule central to memory-efficient SNN training. Beyond BPTT, alternatives have been developed to avoid backpropagation due to biological implausibility and backward gradient propagation. Among these alternatives, Eligibility Propagation (E-PROP) [7] is native to SNNs, while the Forward-Forward algorithm [8] was first introduced for ANNs and later adapted to SNNs [9]. PEPITA [10] follows a similar ANN-originated path. In this work, RATE-PEPITA is proposed as an SNN-compatible, spike-rate-based reformulation of PEPITA. Although these methods were not designed for memory reduction, their learning mechanisms lead to different storage requirements, making them relevant to memory-efficient SNN learning. In memory-constrained settings, forward-learning methods can reduce storage requirements. Although they may introduce accuracy loss, edge-assisted frameworks can mitigate it by boosting weak edge models through server-side aggregation [11].

Long temporal sequences further increase training memory by extending the horizon over which state information must be processed. In time-series learning, windowing has been used to control the temporal scale of long input signals [12]. This work applies the same input-level idea to SNN training: raw temporal inputs are split into shorter segments, reducing the effective sequence length and memory requirements while helping shallow SNNs focus on local temporal patterns.

To analyze these effects, common memory-footprint formulations are developed for BPTT, E-PROP, the SNN adaptation of the Forward-Forward algorithm (FF) [9], and RATE-PEPITA under a unified hardware-agnostic framework. These formulations are used to compare the accuracy–memory trade-offs of the learning rules across diverse real-world datasets.

The main contributions of this paper are as follows:

- RATE-PEPITA is introduced by adapting the ANN-based PEPITA algorithm to SNNs.
- Input-level temporal windowing is evaluated and optimized as a practical strategy for reducing training memory while improving classification accuracy.
- A unified memory-footprint formulation is developed for BPTT, E-PROP, FF, and RATE-PEPITA in SNNs.
- A comprehensive evaluation of memory scaling, reduced precision, and real-world dataset performance is presented to characterize practical trade-offs.

II. RELATED WORK

A central challenge in SNN training is assigning credit through time despite the discrete, non-differentiable nature of spikes. A common solution is to unfold the network over time and use surrogate gradients for BPTT-based optimization [3]. While effective, this requires storing neuron states, spike activity, and intermediate variables across time steps, making memory usage sensitive to sequence length. This has motivated local SNN learning rules such as E-PROP, which uses eligibility traces to summarize recent pre- and postsynaptic activity [7]. E-PROP reduces the need to store the full temporal activity history, but shifts part of the memory burden to synapse-level trace variables.

Backpropagation-free methods proposed for other types of neural networks have also influenced SNN research. For example, Hinton’s Forward-Forward algorithm eliminates backpropagation in ANNs by training each layer using local goodness objectives computed from positive and negative samples, thereby avoiding a conventional backward pass [8]. This approach has been explored in SNNs for robustness to unfamiliar inputs [13] and for supervised classification [9]. PEPITA is another ANN-originated backpropagation-free method, where learning is based on the difference between an original forward pass and an error-perturbed forward pass [10]. Hence, its architecture is simpler than other architectures. This simplicity makes PEPITA a suitable candidate for a memory-efficient SNN adaptation. To our knowledge, our work is the first to adapt PEPITA to SNNs. Since the original PEPITA formulation relies on continuous ANN activations, RATE-PEPITA is introduced in this work as a spike-rate-based adaptation for SNNs. Training memory in SNNs depends not only on the learning rule, but also on the length of the input sequence processed by the network. Prior work has reduced this cost by truncating BPTT, since processing fewer time steps lowers the amount of temporal state that must be stored [4]. In ANN-based time-series learning, input sequences are commonly divided into temporal windows, where the selected window size determines the available temporal context and can affect classification performance [12], [14]. Motivated by these studies, temporal windowing is used here as an input-level preprocessing step for SNN training. Raw analog sequences are split into shorter segments before being processed by the SNN, so each segment defines the number of time steps processed by the network. Each segment is then provided directly to the first LIF layer without spike conversion, following the analog-input setup used in prior SNN training [15]. This corresponds to direct encoding as defined in [16].

Although windowing is well established in ANN-based time-series learning, input-level windowing remains comparatively underexplored in SNN training [17]. Existing studies mainly use windowing for task-specific preprocessing or BPTT truncation, rather than as a broadly applicable training-memory strategy. This motivates our evaluation of temporal windowing as a general, learning-rule-independent memory-control mechanism.

III. PROPOSED METHOD

This section defines the SNN model used in this work and introduces the proposed learning and memory-reduction components.

SNNs are modeled as discrete-time dynamical networks in which membrane potentials evolve over time and spikes are generated by threshold-based firing. Although several spiking neuron models exist, the leaky integrate-and-fire (LIF) model is adopted because it provides a simple and widely used abstraction of temporal integration, leakage, and firing behavior [18].

An SNN with L fully connected spiking layers is considered. For layer l , let d_l denote the number of neurons. At time step t , the membrane potential of layer l is denoted by $u_t^{(l)} \in \mathbb{R}^{d_l}$, while its output is denoted by $z_t^{(l)}$. The network processes an input sequence over T time steps.

Inputs are provided using direct encoding [16]. Thus, the input layer is written as $z_t^{(0)} = x_t$, where $x_t \in \mathbb{R}^{d_0}$ is the continuous-valued input feature vector. For spiking layers $l \geq 1$, $z_t^{(l)} \in \{0, 1\}^{d_l}$ denotes the binary spike output.

At each time step, neuron dynamics follow the discrete-time LIF update

$$u_t^{(l)} = \lambda u_{t-1}^{(l)} + W^{(l)} z_t^{(l-1)} + b^{(l)}, \quad (1)$$

where $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, $b^{(l)} \in \mathbb{R}^{d_l}$, and λ is the membrane leak factor.

Spike outputs are generated as

$$z_t^{(l)} = \Theta(u_t^{(l)} - V_{\text{th}}), \quad (2)$$

where $\Theta(\cdot)$ is the Heaviside step function and V_{th} is the firing threshold. After spike generation, reset-by-subtraction is used, where the threshold is subtracted from the membrane potential to avoid unbounded accumulation.

For classification, final-layer spikes are accumulated over time and used as the output representation.

A. RATE-PEPITA

PEPITA is a backpropagation-free learning algorithm originally proposed for ANNs [10]. It replaces the backward pass with two forward passes: one using the original input and another using an error-perturbed input obtained by projecting the output error back to the input space through a fixed feedback matrix.

RATE-PEPITA is introduced in this work as an SNN-compatible adaptation of PEPITA. It replaces continuous ANN activations with time-averaged activity representations obtained by accumulating layer activities over the simulation window:

$$r_l = \frac{1}{T} \sum_{t=1}^T z_t^{(l)}, \quad l = 0, 1, \dots, L. \quad (3)$$

For $l = 0$, this is the temporal average of the directly encoded analog input, since $z_t^{(0)} = x_t$. For $l \geq 1$, it is the average spike activity of the spiking layers.

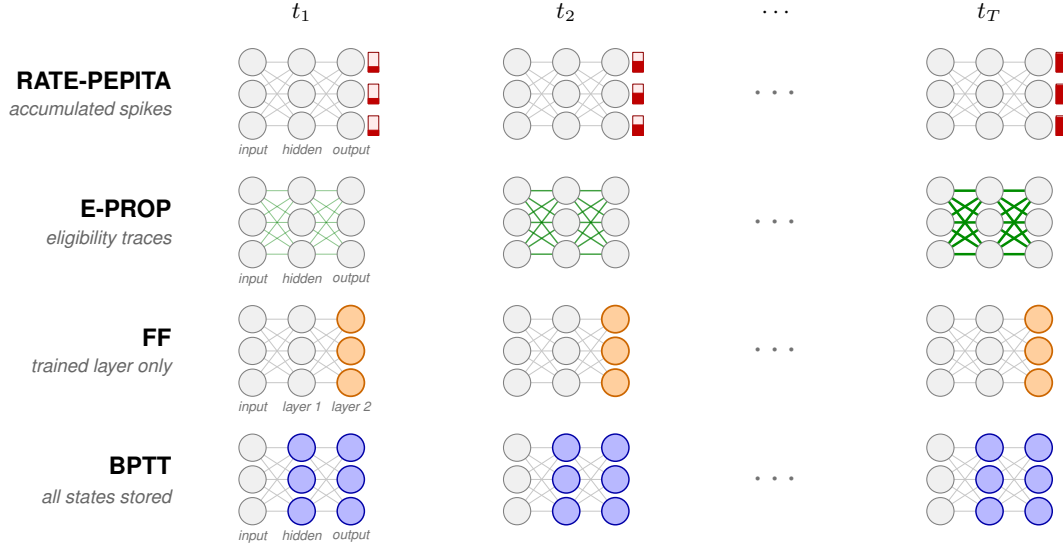


Fig. 1: Memory behavior of four SNN training methods on a time-unrolled network.

In the first pass, the original input sequence is processed and the rate representations r_l are computed. The readout activity is accumulated over time to obtain the output logits, from which the output error is calculated. This error is projected to the input space using a fixed random feedback matrix initialized by sampling uniformly from $[-1, 1]$ and kept untrained, as in the original ANN-based PEPITA paper [10]. The projected error is added to the input at every time step. The perturbed sequence is then processed in the second pass, producing r_l^{err} .

Given learning rate η and output error e , the hidden-layer and output-readout updates are computed as:

$$\begin{aligned} \Delta W^{(l)} &= -\eta (r_l - r_l^{\text{err}}) (r_{l-1}^{\text{err}})^\top, \quad l = 1, \dots, L, \\ \Delta W_o &= -\eta e (r_L^{\text{err}})^\top. \end{aligned} \quad (4)$$

Its memory behavior relative to BPTT, E-PROP, and a recent adaptation of Hinton’s Forward-Forward algorithm to SNNs (FF) [9] is illustrated in Fig. 1 and analyzed in Section III-C. In the figure, the shared architecture is rendered identically across all rows, while colored elements indicate what each method retains during training. RATE-PEPITA accumulates spike counts over time and converts them into spike-rate representations. E-PROP maintains synaptic eligibility traces that build up over time, shown by progressively stronger connection strokes. FF updates only the currently trained layer and uses no output classification head, while BPTT stores the full neuron state at every time step.

B. Temporal Windowing

Temporal windowing is used in this work as an explicit memory-reduction mechanism. Instead of processing the full input sequence of length T , each sequence is divided into shorter temporal segments, referred to as *windows*. Each segment has length S , where $S < T$ for windowed inputs. Thus, under windowing, the effective temporal dimension becomes

S , and memory terms that scale with T are evaluated using $T = S$, reducing the corresponding memory requirements.

Let H denote the hop ratio, so the hop size between consecutive windows is $H \cdot S$. When $H < 1$, windows overlap, increasing temporal continuity and redundancy. When $H = 1$, windows are non-overlapping. When $H > 1$, gaps are introduced, resulting in partial coverage of the original sequence.

Beyond memory reduction, windowing also changes the learning problem. Shorter windows reduce the temporal horizon that the network must model, which can help shallow SNNs focus on localized discriminative patterns and improve temporal resolution. This is particularly useful for datasets where class-relevant information is concentrated in short temporal regions. However, smaller window lengths or lower hop ratios require more training and inference evaluations per sequence. If windows become excessively short, they may also remove long-range dependencies that are important for tasks with slower temporal structure.

Because the optimal window length is dataset-dependent, the Tree-Structured Parzen Estimator (TPE) is used to search over the windowing hyperparameters [19]. TPE models promising and less-promising hyperparameter configurations separately, enabling more efficient exploration than random or grid search.

During inference, each input sequence is partitioned using the selected window length and hop ratio. The network produces a prediction for each window, and the final sequence label is obtained by majority voting over the window-level predictions.

C. Memory Footprint Estimation

To compare the learning rules under a common framework, memory footprint is estimated using a hardware-agnostic flat memory model. The analysis captures algorithmic stor-

age requirements, while accelerator-specific memory hierarchies, caching strategies, and dataflow optimizations are excluded. This allows the memory behavior of BPTT, E-PROP, FF, and RATE-PEPITA to be compared independently of implementation-specific effects. Memory footprint formulation in this work is inspired by [4] and is defined for classification networks. The training memory is decomposed as

$$N_{\text{training}} = N_{\text{input}} + N_{\text{param}} + N_{\text{state}} + N_{\text{intermediate}} + N_{\text{grad}}. \quad (5)$$

Here, N_{input} denotes the input batch, N_{param} denotes weights and biases, N_{state} denotes neuron states maintained during temporal propagation, $N_{\text{intermediate}}$ denotes method-specific quantities used during learning, and N_{grad} denotes parameter gradients. The input memory is given by

$$N_{\text{input}} = BTd_0 \quad (6)$$

where B is the batch size, T is the number of time steps, and d_0 is the input dimension.

The number of trainable parameters is given by

$$N_{\text{param}} = \sum_{l=1}^L (d_{l-1}d_l + d_l) + C \cdot d_L. \quad (7)$$

Here, d_l denotes the dimension of layer l , d_L is the final hidden-layer dimension, and C is the number of classes. The final term represents the classification readout. For FF, this readout is omitted because class information is instead introduced by augmenting the input dimension with C label channels.

The remaining terms depend on the learning method. For BPTT, the membrane potential of each neuron must be stored at every time step to reconstruct the temporal dynamics during backpropagation. In addition, forward-pass quantities such as pre-activations or spike outputs must also be stored for gradient computation. These terms scale identically and can be expressed as

$$N_{\text{state}}^{\text{BPTT}} = N_{\text{intermediate}}^{\text{BPTT}} = BT \sum_{l=1}^L d_l \quad (8)$$

As seen, memory grows linearly with respect to time. This is the main drawback of BPTT: it requires storing the full history of activations. The gradient memory satisfies $N_{\text{grad}}^{\text{BPTT}} = N_{\text{param}}$, since gradients are accumulated for each parameter before being applied.

E-PROP addresses the time dependence of memory by maintaining synaptic eligibility traces that evolve over time. There is no need to store the full activation history. Thus, the state memory becomes

$$N_{\text{state}}^{\text{E-PROP}} = B \sum_{l=1}^L d_l \quad (9)$$

However, eligibility traces must be maintained for each synapse. Therefore, the intermediate storage corresponds to these traces and is given by

$$N_{\text{intermediate}}^{\text{E-PROP}} = B \sum_{l=1}^L d_{l-1}d_l \quad (10)$$

As seen, this term scales with the number of synapses. This creates a trade-off between BPTT and E-PROP: one scales with time, while the other scales with network size. The gradient memory again satisfies $N_{\text{grad}}^{\text{E-PROP}} = N_{\text{param}}$.

In the FF algorithm, layers are trained sequentially rather than jointly. At any given time, only a single layer is updated, while the preceding layers are used to generate its input. Therefore, instead of scaling with the total number of layers, gradient storage is limited to the currently trained layer.

However, forward propagation through the network is still required to compute the input of the currently trained layer. Therefore, the state memory consists of two components: (i) stored activations from previous layers and (ii) the temporal processing of the active layer. This leads to

$$N_{\text{state}}^{\text{FF}} = B \sum_{l=1}^{L-1} d_l + BT \max_{1 \leq l \leq L} d_l \quad (11)$$

where the second term represents the worst-case scenario in which the largest layer is processed over all time steps.

Since FF relies on accumulated spike counts rather than storing full temporal traces, intermediate storage is required only for the active layer. Similarly, gradients are computed and stored only for that layer. Thus,

$$\begin{aligned} N_{\text{intermediate}}^{\text{FF}} &= BT \max_{1 \leq l \leq L} d_l \\ N_{\text{grad}}^{\text{FF}} &= \max_{1 \leq l \leq L} (d_{l-1}d_l + d_l) \end{aligned} \quad (12)$$

Therefore, unlike BPTT, memory usage in FF is determined by the largest layer rather than the full network depth.

RATE-PEPITA is based on computing the difference in spike rates between two phases. This allows the accumulation of spike counts without storing neuron activity at each time step. Spike rates are obtained by dividing the accumulated spike counts by T . Additionally, one extra matrix is required during training to project the output back to the input. Therefore, the state and intermediate memory are given by

$$\begin{aligned} N_{\text{state}}^{\text{RATE-PEPITA}} &= B \sum_{l=1}^L d_l \\ N_{\text{intermediate}}^{\text{RATE-PEPITA}} &= B \sum_{l=1}^L d_l + Cd_0 \end{aligned} \quad (13)$$

Neuron states are required to run the network, while spike counts are accumulated to compute gradients in the second phase. The gradient memory again satisfies $N_{\text{grad}}^{\text{RATE-PEPITA}} = N_{\text{param}}$. This formulation is independent of time and scales only with the total hidden size.

In most practical settings, the differences in memory usage across learning methods are mainly determined by N_{state} and $N_{\text{intermediate}}$, since these terms reflect the storage required by

temporal dynamics and gradient computation. Therefore, the overall memory complexity for each method can be summarized as

$$\begin{aligned}
N_{\text{training}}^{\text{BPTT}} &= O\left(BT \sum_l d_l\right) \\
N_{\text{training}}^{\text{E-PROP}} &= O\left(B \sum_l d_{l-1} d_l\right) \\
N_{\text{training}}^{\text{FF}} &= O\left(BT \max_l d_l\right) \\
N_{\text{training}}^{\text{RATE-PEPITA}} &= O\left(B \sum_l d_l\right)
\end{aligned} \tag{14}$$

Inference memory is identical across methods under the same network architecture, since inference only requires forward execution without gradient-related storage. It can be expressed as

$$\begin{aligned}
N_{\text{inference}} &= N_{\text{input}} + N_{\text{param}} + N_{\text{state}}^{\text{inference}} \\
N_{\text{state}}^{\text{inference}} &= B \sum_{l=1}^L d_l
\end{aligned} \tag{15}$$

Inference memory is significantly smaller than training memory and is independent of the training method.

IV. RESULTS

A. Datasets and Preprocessing

The proposed approach is evaluated on several benchmark datasets, including UCI HAR [20], PAMAP2 [21], MIT-BIH [22], DVS Gesture [23], ESC-50 [24], UrbanSound8K [25], Speech Commands [26], and a large-scale emergency-vehicle audio dataset [27]. These datasets cover audio, human activity recognition, physiological signals, and neuromorphic data. A summary of dataset characteristics is provided in Table I.

TABLE I: Overview of datasets.

Dataset	Input Features	Time Steps	Classes
HAR	9	128	6
PAMAP2	27	128	12
MIT-BIH	1	360	2
DVS Gesture	2048	200	4
ESC-50	64	101	4
UrbanSound8K	64	401	3
Speech Commands	64	101	3
Large-Scale Audio	64	901	2

Audio signals are converted into log-mel spectrograms [28], while sensor-based datasets are represented as normalized multivariate time series. ECG signals from MIT-BIH are segmented into fixed-length windows, and event streams in DVS Gesture are transformed into voxel representations [29].

For some datasets, the task is restricted to a fixed subset of classes to support fair comparison under a unified shallow SNN architecture. Since this work focuses on memory-accuracy trade-offs rather than maximizing full-dataset benchmark accuracy, the same class subsets, preprocessing, architecture, batch size, and evaluation protocol are used for all

learning methods. Unless otherwise stated, all experiments use a one-hidden-layer SNN with 128 hidden neurons, a LIF leak factor of $\lambda = 0.9$, and a firing threshold of $V_{\text{th}} = 1.0$. All methods are trained for 10 epochs with batch size 128, FP32 precision, and random seed 123, without early stopping. BPTT uses a fast-sigmoid surrogate gradient with slope 25. Learner-specific optimizer and learning-rate settings are used due to different update dynamics. Class-balanced splits are applied to avoid bias toward majority classes. All implementations use the `snnTorch` library [1]. The code used for the experiments is available in the accompanying repository [30].

B. Memory Analysis

To compare the memory scaling of different learning algorithms, the HAR dataset is used with multiple configurations, as summarized in Table II. Sequence length, network depth, and hidden size are varied to analyze their impact on memory usage while keeping the dataset fixed.

TABLE II: HAR configurations for memory experiments.

Configuration	Time Steps	Hidden Size	Layers
Baseline	128	128	1
Windowed ($S = 16$)	16	128	1
Deep (2-layer)	128	128	2
Wide Deep (2-layer)	128	1024	2

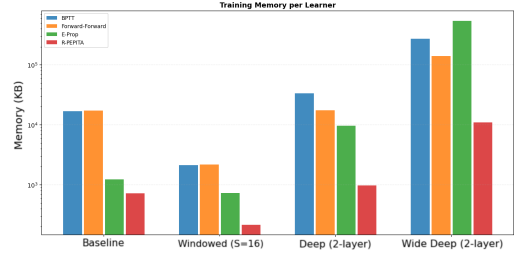
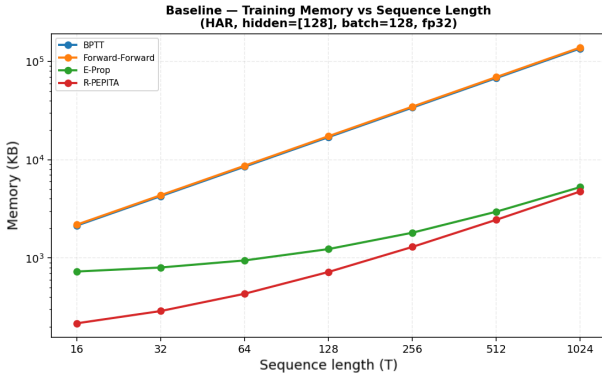


Fig. 2: Estimated training memory requirements across the configurations defined in Table II for each learning algorithm.

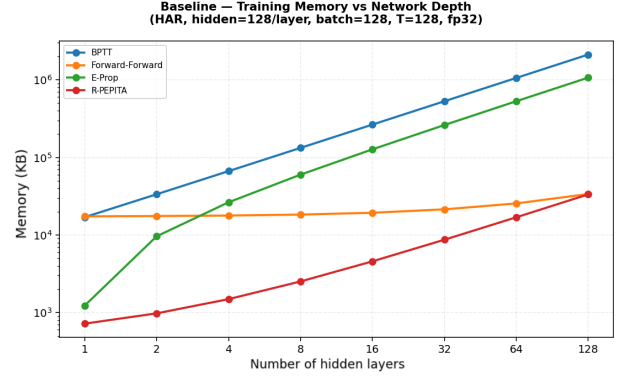
Using these configurations and the memory formulations in Section III-C, the estimated total memory requirements are illustrated in Fig. 2. The most notable behavior is observed for RATE-PEPITA. Across all configurations, it consistently achieves the lowest memory usage. This is because RATE-PEPITA is independent of time steps and does not require maintaining state for each connection. Instead, it only stores a single state per neuron. As a result, memory remains low even when sequence length or network size increases.

E-PROP also shows significantly lower training memory compared to BPTT and FF in the single-layer setting. This is because, in E-PROP, traces are accumulated at each time step, and only the state for each connection needs to be stored and updated. There is no need to store intermediate states across all time steps.

This behavior is further illustrated in Fig. 3a. Reducing the sequence length from 128 to 16 mainly reduces memory for BPTT and FF, which retain a stronger dependence on



(a)



(b)

Fig. 3: Estimated memory scaling of learning algorithms: (a) effect of sequence length on training memory; (b) effect of network depth on training memory.

time. E-PROP and RATE-PEPITA show much weaker time-step dependence, although input storage prevents their total memory from being fully independent of sequence length. The extrapolation to longer sequences shows that this weaker dependence can still become visible as the temporal dimension grows. The wide deep configuration highlights the structural limitation of E-PROP: its per-synapse eligibility traces grow with the number of connections, which increases rapidly with depth and hidden size. Therefore, E-PROP can exceed BPTT in memory usage when connection-level storage dominates over BPTT’s time-unrolled neuron-state storage.

Although BPTT and FF appear similar in the total memory comparison in Fig. 2, their difference becomes clearer as network depth increases. Since FF trains the network layer by layer, adding more hidden layers has a smaller effect on its memory usage. As shown in Fig. 3b, FF therefore remains nearly constant with increasing depth compared with BPTT, making its memory advantage more visible in deeper architectures.

Each method therefore offers a distinct memory trade-off. E-PROP avoids temporal state storage by using synaptic traces, FF reduces memory by updating one layer at a time, and RATE-PEPITA achieves the lowest footprint by avoiding both temporal-history and connection-level storage.

C. Temporal Windowing Analysis

Window lengths are selected using hyperparameter optimization with Optuna [31], employing the TPE-based search strategy. The ranges of possible window lengths and hop ratios are specified as search spaces, and Optuna is used to sample candidate values from these spaces and identify the best-performing configuration. Since the optimal temporal scale depends on the dataset and learning rule, the window length is first optimized with the hop ratio fixed at $H = 1$, and the hop ratio is then tuned using the best-performing window length.

To accelerate the search process, a two-phase strategy is used. In the first phase, a broad search is performed over the predefined search space using 20 trials trained for 5 epochs.

In the second phase, 10 additional trials are trained for 10 epochs within a narrower region centered around the best configuration found in the first phase. The search spaces used in the two phases are defined as

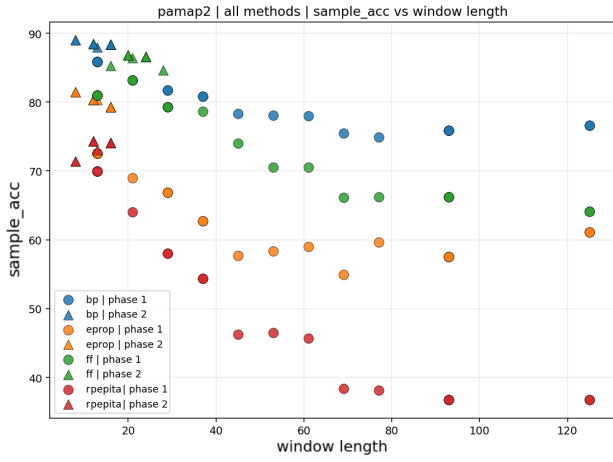
$$\begin{aligned} S_{\text{phase1}} &\in [0.1T, T] & H_{\text{phase1}} &\in [0.25, 1.25] \\ S_{\text{phase2}} &\in [0.75S^*, 1.25S^*] & H_{\text{phase2}} &\in [0.75H^*, 1.25H^*] \end{aligned} \quad (16)$$

where S^* and H^* denote the best window length and hop ratio obtained from the first-phase search, respectively. The first phase provides broad coverage across datasets, while the second phase refines the search around the most promising region.

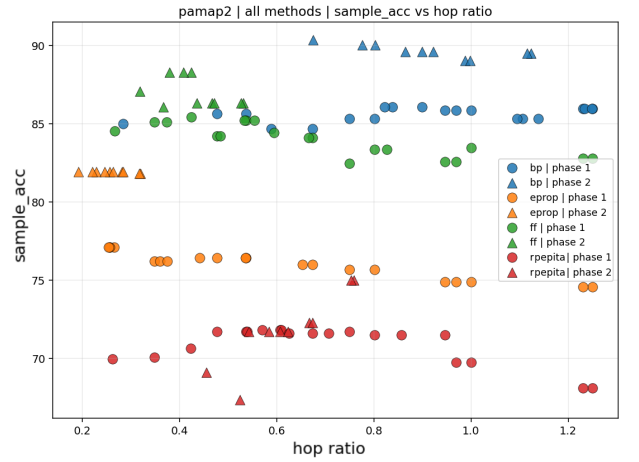
Results of the TPE search on PAMAP2 are shown in Fig. 4. Window length has a strong and consistent impact on accuracy, with shorter windows generally improving all methods; for example, RATE-PEPITA increases by 35%. This suggests that PAMAP2 contains discriminative activity patterns in local temporal regions. By reducing the temporal horizon modeled by the shallow SNN, shorter windows simplify learning and improve temporal resolution. Compared with window length, the hop ratio has a smaller but still meaningful effect; for example, RATE-PEPITA gains about 3% accuracy at $H = 0.7$, showing that hop-ratio tuning can further improve performance after the window length is selected.

The benefit of shorter windows is particularly important for RATE-PEPITA, which relies on accumulated spike-rate representations rather than explicit temporal credit assignment. When the full sequence is used, temporally distinct patterns may be compressed into a coarse rate estimate. Windowing reduces this effect by computing rates over shorter segments, which better preserves local temporal information.

For each dataset and learning rule, the no-window baseline and the best TPE-selected windowed configuration are compared in terms of accuracy and training memory in Table III. Bold entries indicate the best accuracy and lowest memory for each dataset. Overall, temporal windowing often reduces training memory while improving test accuracy. RATE-PEPITA



(a)



(b)

Fig. 4: Effect of temporal windowing parameters on PAMAP2 accuracy: (a) window-length search with fixed hop ratio $H = 1$; (b) hop-ratio search with the window length fixed to the best value found in the window-length search for each learning rule.

TABLE III: Classification accuracy (%) and estimated training memory with and without windowing. For windowed experiments, selected window length (S) and hop ratio (H) are reported; in the no-window setting, S represents the original sequence length. Green cells indicate the highest accuracy for each dataset, while blue cells indicate the lowest memory.

Dataset	Setting	BPTT				FF				E-PROP				RATE-PEPITA			
		S	H	Mem.	Acc.	S	H	Mem.	Acc.	S	H	Mem.	Acc.	S	H	Mem.	Acc.
HAR	No window	128	–	16.6 MB	83.1	128	–	17.0 MB	78.6	128	–	1.2 MB	77.6	128	–	720 KB	62.9
	Windowed	21	0.3	2.7 MB	85.7	21	0.3	2.8 MB	88.6	8	0.7	692 KB	76.5	13	0.5	203 KB	75.5
PAMAP2	No window	128	–	17.7 MB	84.2	128	–	18.5 MB	71.5	128	–	3.5 MB	63.6	128	–	1.8 MB	49.0
	Windowed	8	0.7	1.1 MB	90.4	20	0.4	2.9 MB	88.3	8	0.3	1.9 MB	81.9	12	0.8	331 KB	75.0
Speech Commands	No window	101	–	15.8 MB	91.2	101	–	16.0 MB	82.9	101	–	7.3 MB	84.9	101	–	3.3 MB	63.2
	Windowed	100	1.0	15.7 MB	91.9	92	1.0	14.6 MB	82.9	90	1.0	6.9 MB	86.4	80	1.0	2.7 MB	66.0
ESC-50	No window	101	–	15.8 MB	83.8	101	–	16.0 MB	83.8	101	–	7.3 MB	67.7	101	–	3.3 MB	64.0
	Windowed	10	1.0	1.6 MB	91.9	26	0.8	4.2 MB	93.4	12	0.3	4.5 MB	91.2	10	0.4	518 KB	85.3
Large-Scale Audio	No window	901	–	140.8 MB	97.1	901	–	141.7 MB	92.2	901	–	32.3 MB	96.2	901	–	28.3 MB	86.7
	Windowed	304	1.0	47.6 MB	97.7	24	1.0	3.8 MB	97.8	62	1.0	6.1 MB	97.5	158	1.0	5.1 MB	94.0
UrbanSound8K	No window	401	–	62.7 MB	59.6	401	–	63.3 MB	58.7	401	–	16.7 MB	64.9	401	–	12.7 MB	34.7
	Windowed	224	0.8	35.1 MB	78.7	48	1.0	7.6 MB	82.7	76	1.0	6.5 MB	74.7	64	1.0	2.2 MB	69.3
DVS Gesture	No window	200	–	227.0 MB	97.9	200	–	227.4 MB	96.9	200	–	331.0 MB	86.5	200	–	202.2 MB	76.0
	Windowed	68	1.0	78.6 MB	99.0	100	1.0	114.7 MB	99.0	16	1.0	146.1 MB	97.9	36	1.6	38.2 MB	93.8
MIT-BIH	No window	360	–	45.2 MB	71.5	360	–	45.5 MB	74.8	360	–	312 KB	59.5	360	–	312 KB	50.0
	Windowed	260	1.0	32.6 MB	71.8	196	1.0	24.8 MB	76.0	308	1.0	286 KB	70.8	88	0.9	176 KB	58.0

generally achieves the lowest memory usage and benefits considerably from windowing; the exception is Large-Scale Audio, where FF has the smallest footprint because TPE selects a much shorter window. For example, on PAMAP2, RATE-PEPITA accuracy increases from 49% to 75%, while memory decreases from 1.8 MB to 331 KB. Similar trends on UrbanSound8K and DVS Gesture show that windowing makes RATE-PEPITA more competitive while preserving its low-memory advantage. MIT-BIH highlights a limitation of RATE-PEPITA: rate-only representations can be insufficient when fine-grained waveform morphology is discriminative.

The results also highlight the trade-offs between learning rules. FF often achieves the best accuracy, likely because its label-conditioned inference evaluates each candidate class separately, providing an additional class-wise comparison mech-

anism. However, this requires per-class inference, increasing the inference cost as the number of classes grows. E-PROP provides a useful accuracy–memory balance when temporal activation storage dominates, but its advantage decreases when synapse-level eligibility traces become costly, as seen on DVS Gesture. RATE-PEPITA avoids this drawback because it does not store synaptic traces, making it the most suitable option when training memory is the main constraint.

Finally, the selected window parameters show that the optimal temporal scale is dataset-dependent. Shorter windows often improve accuracy on PAMAP2, ESC-50, UrbanSound8K, Large-Scale Audio, and DVS Gesture, while Speech Commands benefits from longer temporal context. The selected hop ratios show that overlap is useful for some datasets, especially HAR, PAMAP2, and ESC-50, whereas non-overlapping win-

dows are often sufficient for others. Thus, both window length and hop ratio should be treated as important hyperparameters rather than fixed preprocessing choices.

D. Precision Analysis

Precision is evaluated as an additional memory-saving factor, since the memory footprint is directly affected by stored-value size. The FP32, FP16, and BF16 formats are compared. A narrower dynamic range is introduced by FP16, whereas the FP32 exponent range is preserved by BF16 with reduced mantissa precision [32].

TABLE IV: Effect of precision on accuracy for the HAR dataset

Dataset	Precision	BPTT	E-PROP	FF	RATE-PEPITA
HAR	FP32	84.06	77.63	78.58	63.69
	FP16	83.75	74.82	80.08	63.22
	BF16	81.64	75.43	74.92	60.84

Reduced precision can nearly halve memory requirements with limited accuracy degradation [33]. In the memory formulations of Section III-C, this corresponds to using 2 bytes per stored value instead of 4 bytes. The resulting accuracy trends for the HAR dataset are reported in Table IV.

As can be seen, there is a small drop in accuracy for each method. In some cases, slight improvements are also observed due to differences in learning dynamics caused by the change in precision. These results suggest that reduced-precision training can provide substantial memory savings with limited accuracy degradation.

V. CONCLUSION

This work analyzed the proposed RATE-PEPITA method, E-PROP, FF, and BPTT under a unified memory framework. RATE-PEPITA achieves the lowest estimated training-memory usage by relying on accumulated spike-rate representations rather than temporal histories or synaptic traces, making it most suitable when memory is the primary constraint. E-PROP and FF also reduce memory through eligibility traces and layer-local training, respectively, while BPTT remains the most memory-demanding due to time-unrolled state storage. Although RATE-PEPITA may trade accuracy for reduced storage, it can enable training under strict memory constraints where BPTT is impractical. Future work should validate these trade-offs with real-hardware memory, runtime, and energy measurements.

The results also show that temporal windowing reduces memory and, when selected in a data-directed manner, can help backpropagation-free methods match or outperform BPTT on several datasets.

Overall, memory-efficient SNN training requires joint optimization of the learning rule, temporal window, and precision. RATE-PEPITA offers a practical direction for memory-constrained SNN training, especially when combined with temporal windowing.

REFERENCES

- [1] J. K. Eshraghian et al., "Training Spiking Neural Networks Using Lessons From Deep Learning," *Proc. IEEE*, 2023.
- [2] K. Roy et al., "Towards spike-based machine intelligence with neuro-morphic computing," *Nature*, 2019.
- [3] E. O. Neftci et al., "Surrogate Gradient Learning in Spiking Neural Networks," arXiv:1901.09948, 2019.
- [4] W. Guo et al., "Efficient training of spiking neural networks with temporally-truncated local BPTT," *Front. Neurosci.*, 2023.
- [5] Y. D. Kwon et al., "TinyTrain: Resource-Aware Sparse Training of DNNs at the Edge," arXiv:2307.09988, 2024.
- [6] J. Lin et al., "On-Device Training Under 256KB Memory," arXiv:2206.15472, 2022.
- [7] G. Bellec et al., "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nat. Commun.*, 2020.
- [8] G. Hinton, "The Forward-Forward Algorithm: Some Preliminary Investigations," arXiv:2212.13345, 2022.
- [9] M. Ghader et al., "Backpropagation-free Spiking Neural Networks with the Forward-Forward Algorithm," *Sci. Rep.*, 2026.
- [10] G. Dellaferrera and G. Kreiman, "Error-driven Input Modulation," in *Proc. ICML*, 2022.
- [11] İ. Sikkokur et al., "EdgeConvEns: Convolutional Ensemble Learning for Edge Intelligence," *IEEE Access*, vol. 12, pp. 168314–168327, 2024.
- [12] Baños et al., "Window Size Impact in Human Activity Recognition," *Sensors*, 2014.
- [13] E. B. Terres-Escudero et al., "Fully-spiking neural networks in open-world scenarios using forward-only learning," arXiv:2407.14097, 2024.
- [14] Jaén-Vargas et al., "Effects of Sliding Window Variation in the Performance of Acceleration-Based Human Activity Recognition Using Deep Learning Models," *PeerJ Computer Science*, 2022.
- [15] R. Zimmer et al., "Supervised training of convolutional spiking neural networks with PyTorch," arXiv:1911.10124, 2019.
- [16] E. Lemaire et al., "An Analytical Estimation of Spiking Neural Networks Energy Efficiency," in *Proc. ICONIP*, 2022.
- [17] S. Kshirasagar et al., "Impact of sliding window variation and neuronal time constants on acoustic anomaly detection using recurrent spiking neural networks in automotive environment," *Algorithms*, 2024.
- [18] A. N. Burkitt, "A review of the integrate-and-fire neuron model," *Biol. Cybern.*, 2006.
- [19] J. Bergstra et al., "Algorithms for hyper-parameter optimization," in *Proc. NeurIPS*, 2011.
- [20] D. Anguita et al., "A public domain dataset for human activity recognition using smartphones," in *Proc. ESANN*, 2013.
- [21] A. Reiss and D. Stricker, "Introducing a new benchmarked dataset for activity monitoring," in *Proc. ISWC*, 2012.
- [22] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH Arrhythmia Database," *IEEE Eng. Med. Biol. Mag.*, 2001.
- [23] A. Amir et al., "A low power, fully event-based gesture recognition system," in *Proc. CVPR*, 2017.
- [24] K. J. Piczak, "ESC: Dataset for environmental sound classification," in *Proc. ACM Multimedia*, 2015.
- [25] J. Salamon et al., "A dataset and taxonomy for urban sound research," in *Proc. ACM Multimedia*, 2014.
- [26] P. Warden, "Speech Commands: A dataset for limited-vocabulary speech recognition," arXiv:1804.03209, 2018.
- [27] M. Asif et al., "Large-scale audio dataset for emergency vehicle sirens and road noises," *Sci. Data*, 2022.
- [28] B. McFee et al., "librosa: Audio and music signal analysis in Python," in *Proc. SciPy*, 2015.
- [29] D. Gehrig et al., "End-to-end learning of representations for asynchronous event-based data," in *Proc. ICCV*, 2019.
- [30] E. Keskin, "SNN_Learning_Methods," GitHub repository, 2026. [Online]. Available: https://github.com/erto2000/SNN_Learning_Methods
- [31] T. Akiba et al., "Optuna: A next-generation hyperparameter optimization framework," in *Proc. KDD*, 2019.
- [32] D. Kalamkar et al., "A study of BFLOAT16 for deep learning training," arXiv:1905.12322, 2019.
- [33] P. Micikevicius et al., "Mixed precision training," in *Proc. ICLR*, 2018.